Quack Quack What? — Database Schema Document

Version: 1.0
Date: March 2026
Author: Taashi Manyanga
Trademark: © Taashi Manyanga 2026

Overview

The application uses PostgreSQL as its primary data store, accessed via Drizzle ORM. The schema is defined in TypeScript and pushed to the database using `drizzle-kit push`.

Entity Relationship Diagram

See: `diagrams/database-erd.mmd`

Tables

1. `users`

Stores authenticated user profiles. Created/updated on each OIDC login via the `upsertUser` function.

Column	Type	Constraints	Description
`id`	VARCHAR	PK, DEFAULT gen_random_uuid()	User identifier (OIDC subject)
`email`	VARCHAR	UNIQUE, NULLABLE	User's email address
`first_name`	VARCHAR	NULLABLE	First name from OIDC claims
`last_name`	VARCHAR	NULLABLE	Last name from OIDC claims
`profile_image_url`	VARCHAR	NULLABLE	Avatar URL from OIDC claims
`created_at`	TIMESTAMP	DEFAULT now()	Account creation time
`updated_at`	TIMESTAMP	DEFAULT now()	Last profile update time

Notes: - This table is mandatory for Replit Auth — do not drop it. - The `id` field maps to the OIDC `sub` claim.

---

2. `sessions`

Server-side session storage used by `connect-pg-simple`.

Column	Type	Constraints	Description
`sid`	VARCHAR	PK	Session identifier
`sess`	JSONB	NOT NULL	Serialized session data
`expire`	TIMESTAMP	NOT NULL, INDEXED	Session expiry time

Indexes: - `IDX_session_expire` on `expire` column

Notes: - This table is mandatory for Replit Auth — do not drop it. - Sessions expire after 7 days (configurable in `replitAuth.ts`).

3. `evaluations`

Stores all skill evaluation results with scores and detailed findings.

Column	Type	Constraints	Description
`id`	VARCHAR	PK, DEFAULT gen_random_uuid()	Evaluation identifier
`user_id`	VARCHAR	NULLABLE	Owner user ID (null for guest evaluations)
`source`	TEXT	NOT NULL	Source identifier (repo URL, filename, etc.)
`source_type`	TEXT	NOT NULL	Source type: "github", "url", "local", "paste"
`skill_code`	TEXT	NULLABLE	The raw skill source code analyzed
`safety_score`	INTEGER	NOT NULL	Safety score (0-100)
`quality_score`	INTEGER	NOT NULL	Quality score (0-100)
`security_score`	INTEGER	NOT NULL	Security score (0-100)
`overall_score`	INTEGER	NOT NULL	Weighted overall score (0-100)
`summary`	TEXT	NOT NULL	Human-readable evaluation summary
`safety_findings`	JSON	NOT NULL	Array of safety finding strings
`quality_findings`	JSON	NOT NULL	Array of quality finding strings
`security_findings`	JSON	NOT NULL	Array of security finding strings
`created_at`	TIMESTAMP	NOT NULL, DEFAULT now()	Evaluation timestamp

Notes: - `user_id` is null for guest evaluations (no foreign key constraint) - Findings are stored as JSON arrays of strings - The `overall_score` is pre-computed: `safety×0.4 + security×0.3 + quality×0.3`

---

4. `provider_configs`

Stores the configured LLM provider credentials and settings.

Column	Type	Constraints	Description
`id`	VARCHAR	PK, DEFAULT gen_random_uuid()	Config identifier
`provider`	TEXT	NOT NULL	Provider name: openai, anthropic, foundry, vertexai, bedrock
`api_key`	TEXT	NULLABLE	API key or bearer token
`model`	TEXT	NULLABLE	Model identifier (e.g., gpt-4o)
`target_uri`	TEXT	NULLABLE	Custom endpoint URI (for AI Foundry)
`region`	TEXT	NULLABLE	AWS region (for Bedrock)
`project_id`	TEXT	NULLABLE	GCP project ID (for Vertex AI)
`location`	TEXT	NULLABLE	GCP location (for Vertex AI)
`access_key_id`	TEXT	NULLABLE	AWS access key (for Bedrock)
`secret_access_key`	TEXT	NULLABLE	AWS secret key (for Bedrock)

Notes: - Only one config record exists at a time (upsert pattern) - API keys are stored in plaintext in the database (encrypted at rest by PostgreSQL) - The GET endpoint masks sensitive fields (returns `hasApiKey: true/false` instead)

Drizzle Schema Types

Insert Schemas (Zod)

```
insertProviderConfigSchema  // Omits: id
insertEvaluationSchema      // Omits: id, createdAt
```

TypeScript Types

```
type ProviderConfig = typeof providerConfigs.$inferSelect
type InsertProviderConfig = z.infer<typeof insertProviderConfigSchema>
type Evaluation = typeof evaluations.$inferSelect
type InsertEvaluation = z.infer<typeof insertEvaluationSchema>
type User = typeof users.$inferSelect
type UpsertUser = typeof users.$inferInsert
```

Storage Interface

```
interface IStorage {
  getProviderConfig(): Promise<ProviderConfig | undefined>
  upsertProviderConfig(config: InsertProviderConfig): Promise<ProviderConfig>
  createEvaluation(evaluation: InsertEvaluation): Promise<Evaluation>
  getEvaluation(id: string): Promise<Evaluation | undefined>
  getEvaluations(): Promise<Evaluation[]>
  getEvaluationsByUser(userId: string): Promise<Evaluation[]>
}
```
